



AWS Idle Identifier

AWS EC2 & RDS Cost Optimization Solution

Technical Presentation • 2026

Executive Summary

What is AWS Idle Identifier?

A comprehensive Python-based application that monitors and optimizes AWS infrastructure costs by identifying idle and underutilized EC2 instances and RDS databases.



EC2 & RDS Support

Analyzes EC2 instances and RDS databases



Intelligent Idle Detection

Uses CPU, memory, network, and I/O metrics



Cost Analysis

Calculates potential savings with hourly/monthly projections



Interactive Dashboard

Streamlit-based web interface

40%

Max Cost Savings

2

AWS Services

5+

Database Support

Problem Statement

The Cloud Cost Challenge



Common Issues

- Development environments left running overnight
- Oversized EC2 instances for actual workloads
- Oversized RDS databases for actual usage
- RDS instances with zero connections



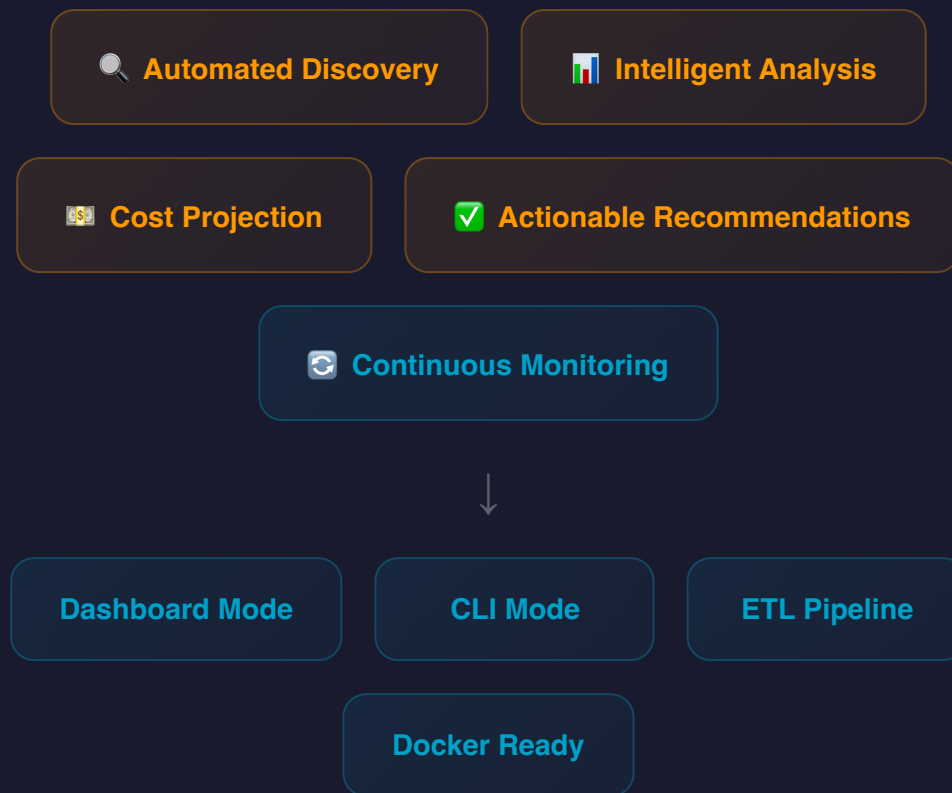
Impact

- 20-30% of cloud spend is wasted
- Manual identification is time-consuming
- Error-prone review processes
- Lack of visibility
- Accumulating costs over time

Solution: AWS Idle Identifier provides automated, continuous monitoring to identify idle EC2 and RDS resources and calculate precise cost savings.

Solution Overview

How AWS Idle Identifier Helps



System Architecture

High-Level Architecture Overview



Key Features

Core Capabilities



Multi-Service Analysis

SERVICE	METRICS	IDLE CRITERIA
RDS	CPU, Connections, IOPS	CPU < 5%, 0 connections
EC2	CPU, Network I/O, Disk	CPU < 5%, minimal traffic



Intelligent Detection

- Multi-indicator scoring system
- Configurable thresholds via environment
- Severity levels: LOW, MEDIUM, HIGH, CRITICAL
- Grace period for recent activity (10%)
- Real-time pricing from AWS API
- Hourly, monthly, annual projections

Technology Stack

Tools and Technologies Used



Core Technologies

CATEGORY	TECHNOLOGY
Language	Python 3.9+
AWS SDK	boto3 >=1.34.0
Web Framework	Streamlit >=1.30.0
Data Processing	pandas >=2.1.0
Visualization	Plotly >=5.18.0
Database ORM	SQLAlchemy >=2.0.0



Database Support

- PostgreSQL (recommended for production)
- MySQL
- Oracle
- MSSQL
- SQLite (default for development)



Security

- bcrypt for password hashing
- Bandit for code analysis
- Safety for dependency scanning

Implementation

Local Installation & Docker

Local Installation

```
# Clone and setup
git clone
cd aws-idle-identifier

# Create virtual environment
python -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure
cp .env.example .env

# Run dashboard
streamlit run main.py
```

Docker Deployment

```
# Build and run
docker-compose up -d

# Access points
Dashboard: http://localhost:8501
pgAdmin: http://localhost:5050
```

CLI Usage

```
# Check all RDS
python main.py --service rds --check

# Check specific EC2
python main.py --service ec2 --install
```


Configuration Options

Environment Variables

VARIABLE	DESCRIPTION	DEFAULT
DATABASE_URL	SQLAlchemy connection string	SQLite
AWS_ACCESS_KEY_ID	AWS Access Key	Required
AWS_SECRET_ACCESS_KEY	AWS Secret Key	Required
AWS_DEFAULT_REGION	AWS Region	us-east-1
CPU_IDLE_THRESHOLD	CPU % threshold	5.0
DB_CONNECTIONS_IDLE_THRESHOLD	DB connections	1.0
NETWORK_IDLE_THRESHOLD	Network bytes/sec	1000

Use Case Scenarios

Practical Applications

01

Dev Environment Cleanup

Detect EC2 instances running over weekends with zero CPU activity

\$500-2,000/month savings

02

Right-sizing Databases

Identify oversized RDS instances with consistently low utilization

30-50% cost reduction

03

Idle EC2 Detection

Find EC2 instances with minimal CPU and network activity

\$100-500/month per instance

04

Database Optimization

Identify RDS instances with zero connections

\$200-1000/month per DB

Benefits and Cost Savings

ROI and Quantified Benefits

20-40%

Idle Resource
Reduction

15-30%

Instance Cost
Reduction

10hrs

Manual Work
Saved/Week

Example ROI Calculation

Scenario: 50 EC2, 20 RDS

Current Monthly Spend:
\$15,000

Idle Resources: 30%

Potential Monthly Savings:
\$4,500

Annual Savings: \$54,000

Intangible Benefits

- Improved resource governance
- Better capacity planning
- Reduced alert fatigue
- Environment accountability
- Consolidated cost visibility

Security Considerations

IAM Permissions and Best Practices

Required IAM Permissions

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "ec2:Describe*",
      "rds:Describe*",
      "rds:ListTagsForResource",
      "cloudwatch:GetMetricStatistics",
      "pricing:GetProducts"
    ],
    "Resource": "*"
  }]
}
```

Best Practices

- **Use IAM roles** instead of access keys when possible
- **Rotate credentials** regularly
- **Least privilege** - grant only required permissions
- **Use VPC endpoints** for AWS API access
- **Secure database files** with chmod 600
- **Enable encryption** at rest
- **Enable CloudTrail** for audit logging

Prerequisites and Limitations

Prerequisites

- Python 3.9 or higher
- AWS Account with billing access
- IAM User/Role with read-only permissions
- Database (SQLite default, PostgreSQL recommended)
- Internet access to AWS APIs

Limitations

- **No auto-termination:** Tool recommends, does NOT delete
- **Cost data delay:** Slight delay in pricing data
- **Region scope:** Must run per-region or specify all
- **API rate limits:** Subject to AWS throttling
- **Historical data:** Limited to 90 days (CloudWatch)

Troubleshooting

Common Issues and Solutions

Database Connection

Error: "Unable to connect to database"

```
# Verify DATABASE_URL  
echo $DATABASE_URL
```

AWS API Throttling

Error: "Rate exceeded"

- Adjust MAX_CONCURRENT_WORKERS
- Implement exponential backoff
- Request limit increase from AWS

Missing Pricing Data

Error: "No pricing found"

```
# Run pricing loader  
python -m etl.pricing_loader
```

ETL Lock Errors

Error: "ETL process is locked"

```
# Force unlock  
python -c "from etl.orchestrator  
import ETLOrchestrator;  
e = ETLOrchestrator();  
e.force_unlock()"
```

Architecture Decisions

Key Design Principles



Dual-Mode Operation

ETL mode for dashboard (cached data), Direct API mode for CLI (real-time). Automatic detection based on execution context.



Database Agnostic

SQLAlchemy for abstraction. Support for 5 database backends. Automatic schema creation.



Separation of Concerns

ETL layer for data collection, Analysis layer for processing, UI layer for presentation.



Pluggable Services

Base manager class with service-specific implementations. Easy to extend for new AWS services.

Future Enhancements

Planned Features

Coming Soon

- **Lambda Analysis** - Cost optimization for serverless
- **Reserved Instance Advisor** - RI recommendations
- **Savings Plans Analysis** - Savings Plan recommendations
- **Email Alerts** - Proactive idle notifications
- **Slack Integration** - Team notifications

Community Requests

- Additional AWS service support
- Custom threshold profiles
- Export to CSV/Excel
- API endpoint for integrations
- Multi-account aggregation

Conclusion

Summary



Automated Discovery

Eliminates manual review with continuous monitoring



Cost Effective

Potential 20-40% savings on cloud spend



Production Ready

Docker deployment with enterprise features

AWS Idle Identifier provides comprehensive solutions for identifying idle EC2 and RDS resources, calculating precise cost savings, and providing actionable recommendations.

Next Steps for Adoption

How to Get Started

Immediate Actions

- 1 Proof of Concept - Deploy with SQLite
- 2 Configure single AWS environment
- 3 Run initial scan

Production Steps

- 1 Set up PostgreSQL database
- 2 Configure multi-environment credentials
- 3 Set up scheduled ETL runs
- 4 Add alerting and review process

Questions?

Thank you for your attention



srathod@mdsol.com



Documentation: [README.md](#)

